

TECHNICAL WHITEPAPER

Vibing Farmer

Autonomous Yield Optimization via Permission-Bounded AI Agents

Abstract

Vibing Farmer automates USDC yield farming on Ethereum through permission-bounded AI agents. Each agent operates under a cryptographic permission defined by the user and enforced by the smart contract, specifying a target vault, a spending cap, and an expiry timestamp. Venice AI generates the allocation strategy from live DeFiLlama APY data; 1Shot relays each transaction gas-free via EIP-7710 delegation redemption. Users grant one scoped permission and the agent handles the rest swap, approve, deposit, and continuous monitoring.

The architecture combines three Ethereum standards: EIP-7702 for smart account delegation, ERC-7715 for permission scoping, and EIP-7710 for gasless delegation redemption with a multi-agent coordination layer and an on-chain strategy attestation mechanism. AgentVaultDepositor.sol, deployed and verified on Sepolia, enforces every constraint with custom errors rather than silent failures and tracks cumulative agent spending on-chain per agentId. 57 Foundry tests cover the full permission lifecycle at 93.3% branch coverage.

1. Problem Statement

Manual yield farming requires constant attention. Moving USDC from Aave to Morpho as APYs shift requires a sequence: check rates, calculate allocation, initiate swap, sign approve, sign deposit and pay \$3-15 in gas per rebalancing event. At a 0.5% APY difference between venues, a \$10,000 position breaks even on gas costs only after months. The math rarely favors the ordinary user.

Vault aggregators like Yearn and Beefy remove execution friction but replace user agency with protocol custody. Users surrender assets to a contract they do not control, receiving shares in a vault whose allocation strategy they cannot inspect or modify. There is no mechanism to say: allocate 60% to Aave, cap at \$1,000, revoke after 24 hours.

DIY automation personal bots, keeper contracts requires engineering skill, private key exposure on a remote server, and no built-in spending limits. A compromised bot means a compromised wallet.

The missing piece is a system where execution is autonomous but boundaries are cryptographically defined and user-controlled. This agent may deposit up to \$1,000 into vault X, before timestamp Y, and nothing else. The contract enforces this unconditionally not with a warning, but with a revert. The AI finds the optimal vault; the blockchain enforces the boundary.

Approach	Strength	Tradeoff
Manual farming	Full user control	Constant attention, \$3-15 gas per action
Vault aggregators	No execution friction	Custodial, fixed strategy, no user control
DIY automation	Custom logic	Private key exposure, no spending limits
Vibing Farmer	Bounded autonomy	Non-custodial, cryptographically scoped, gasless

2. Solution Overview

Vibing Farmer addresses the trilemma with three composable primitives, none of which require custody.

Pillar 1 - Scoped Autonomy (EIP-7702 + ERC-7715)

EIP-7702, live on Ethereum since the Pectra upgrade in March 2025, lets any EOA temporarily set a delegation code and upgrade to a smart account at the same address. No asset migration. No new address. ERC-7715 layered on top lets the user grant a typed permission specifying which function, on which contract, with what spending limit, expiring when.

AgentVaultDepositor.sol validates every agent call against the stored permission before execution. Amount exceeds cap: revert. Vault address does not match: revert. Permission expired: revert. The contract is the enforcement mechanism not the AI, not the relayer.

Pillar 2 - Gasless Execution (1Shot + EIP-7710)

1Shot operates a permissionless relayer that redeems EIP-7710 delegations on behalf of agents. The user pays \$0 in gas. This makes micro-optimizations economically viable: rebalancing a \$500 position when APY drifts 0.3% is free. No API key required.

Pillar 3 - Private Strategy Generation (Venice AI)

Venice AI runs llama-3.3-70b in a trusted execution environment with a documented zero-data-retention policy. The agent sends live DeFiLlama APY data and Tavily market context; Venice returns vault allocations and reasoning. That reasoning is hashed (keccak256) and attested on-chain via StrategyAttested so the allocation decision is a verifiable artifact tied to a specific block.

3. Architecture

3.1 System Overview

```

User Wallet (EOA)
|
| EIP-7702: set delegation code
v
Smart Account (same address, no asset migration)
|
| ERC-7715: grant scoped permission per agentId
v
AgentVaultDepositor.sol <--- StrategyAttested event
|
| (keccak256 reasoning hash on-chain)
|
+--- Orchestrator Agent (browser JS)
|
|   +--- Worker-1 (Conservative scope)
|   |
|   |   +--> MockVault A (aave-v3 simulation)
|   |
|   +--- Worker-2 (Balanced scope)
|   |
|   |   +--> MockVault B (morpho-blue simulation)
|   |
|   +--- Worker-N (Promise.allSettled -- parallel)
|
+--- 1Shot Relay (EIP-7710) ---> Gas: $0 for user
|
|   +--> Ethereum Sepolia
|
|   +--> Events: AgentStarted, SwapExecuted,
|               ApproveExecuted, DepositExecuted,
|               AgentCompleted, StrategyAttested

```

3.2 Smart Contract Layer

AgentVaultDepositor.sol is the on-chain enforcer for the agent system. It holds no user funds and executes on behalf of users who have granted agent permissions via ERC-7715. Every state-changing function is guarded by ReentrancyGuard and follows the checks-effects-interactions pattern throughout.

The core data structure AgentPermission is mapped per user address and agentId:

```

struct AgentPermission {
    address vault;
    uint256 maxAmount;
    uint256 usedAmount;    // cumulative; tracked on-chain, not a caller param
    uint256 expiresAt;
    bool    active;
    bool    allowWithdraw; // opt-in, default false
    bool    allowHarvest;  // opt-in, default false
}

mapping(address => mapping(bytes32 => AgentPermission)) public agentPermissions;

```

usedAmount accumulates with each executeAgentDeposit call. Agents cannot reset or bypass this counter. Withdraw and harvest capabilities require explicit opt-in via setAgentCapabilities; both default to false, so the minimal permission is deposit-only.

StrategyAttested emits a keccak256 hash of the Venice AI reasoning alongside the vault protocol name and allocated amount. Custom errors replace require strings throughout, saving gas and providing precise revert identifiers for off-chain monitoring.

EVENTS EMITTED BY AGENTVAULTDEPOSITOR.SOL

Event	Description
AgentStarted	Agent begins execution for a specific vault
SwapExecuted	Token swap completed (amountIn, amountOut)
ApproveExecuted	ERC-20 approval to vault completed
DepositExecuted	Vault deposit completed (amount, sharesReceived)
AgentCompleted	Full lifecycle complete for one agentId
AgentFailed	Execution reverted with reason string
WithdrawExecuted	Agent withdrawal executed (opt-in only)
HarvestExecuted	Rewards claimed (opt-in only)
StrategyAttested	Venice AI reasoning hash anchored on-chain

DEPLOYED CONTRACTS (ETHEREUM SEPOLIA)

Contract	Protocol	Address (Sepolia)
AgentVaultDepositor	Core enforcer	0xf1441BBC2fa6D37Ce7A5f6254a6A443B281d38f4
MockVault A	aave-v3 sim	0x735f3a63D5be965E6B7564a2befeca0E316d09Ad
MockVault B	morpho-blue sim	0x79007794Eb31B6a8439C38B604827012DBc0D771
MockVault C	pendle-v2 sim	0xAABfc44939E6437446E6FBD4A4e3816C877e371C
MockVault D	fluid sim	0xdef19fED6Da53D3757779d27b9A2640547c30b6F

3.3 AI Strategy Layer

Venice AI produces allocation recommendations in structured JSON using response_format: { type: 'json_object' } with venice_parameters: { include_venice_system_prompt: false }. The prompt contains live DeFiLlama APY/TVL data, Tavily market context, and user parameters (total

amount, risk level, target vault count). Venice returns vault selection, allocation percentages, and a reasoning field. Timeout is 10 seconds; a DeepSeek proxy serves as fallback if Venice is unavailable.

Before any execution, the agent calls `attestStrategy()` on `AgentVaultDepositor`. This emits `StrategyAttested` with `strategyHash = keccak256(abi.encodePacked(reasoning, timestamp))`, `vaultProtocol`, and `allocatedAmount`. Every deposit is traceable to the specific AI decision that triggered it. Venice's zero-data-retention policy means the only persistent record outside the user's browser is this on-chain attestation hash.

3.4 Agent Coordination (A2A)

The Orchestrator Agent runs in the browser as a JavaScript module. It receives the Venice AI allocation plan, spawns one Worker Agent per vault, and executes via `Promise.allSettled()` not `Promise.all()`. One vault failure does not abort the others.

Each Worker Agent follows a fixed sequence: grant ERC-7715 permission, build the 1Shot relay transaction, submit to `AgentVaultDepositor.executeAgentDeposit`, confirm the `DepositExecuted` event, and append a memory file entry. Memory files are append-only local JSON logs surfaced in the `vis.js` agent graph on node click.

A background Web Worker monitors active positions every 60 seconds: current APY, accrued rewards, permission expiry, estimated rebalancing cost. It emits to the main thread for live graph updates but does not trigger on-chain actions. Execution always requires the explicit `executeAgentDeposit` call path.

Each worker permission includes a `parentDelegation` field referencing the Orchestrator's root permission. This chain is verifiable on-chain: every vault deposit traces back to the original user approval.

3.5 Permission Model

```
// ERC-7715 permission structure (per agent, per vault)
{
  target:      <vault_address>,      // only this contract
  selector:    deposit(uint256),      // only this function
  maxAmount:   <user_defined>,       // spending cap (enforced on-chain)
  expiresIn:   86400,                 // 24h default, user-configurable
  revocable:   true                   // user can cancel at any time
}

// On-chain validation in AgentVaultDepositor.executeAgentDeposit:
if (!perm.active)                revert PermissionNotActive();
if (block.timestamp >= perm.expiresAt) revert PermissionExpired();
if (vault != perm.vault)          revert VaultMismatch();
if (perm.usedAmount + amount > perm.maxAmount) revert AmountExceedsPermission();
```

An agent cannot transfer funds to arbitrary addresses. The on-chain enforcer validates every call against the stored permission before execution. Even fully compromised AI output cannot move funds outside the

approved vault the contract will revert.

4. Security Model

4.1 Threat Model

The permission model is designed around a specific set of adversarial inputs. Each is mitigated at the contract layer, not the application layer.

Threat	Mitigation
Agent fund theft	Permission scoped to one vault address. VaultMismatch revert on any other target.
Overspending	usedAmount tracked on-chain per agentId. AmountExceedsPermission revert when cap hit.
Stale permissions	24h expiry by default. PermissionExpired revert after timestamp. User can revoke immediately.
AI hallucination	Even malformed Venice output cannot exceed the on-chain scope. Contract is the enforcer, not the AI.
Cross-agent access	agentPermissions[user][agentId] keyed per agentId. Agent 1 cannot access Agent 2's vault.
Reentrancy	ReentrancyGuard on all execute* entry points. CEI pattern enforced throughout.

4.2 Defense in Depth

Layer	Scope	Control
1	Contract	ERC-7715 permission enforcement. Every violation reverts with a custom error. No silent failures.
2	Contract	ReentrancyGuard + CEI pattern on all state-changing functions.
3	Application	CORS allowlist + rate limiting on Venice AI proxy. Blocks SSRF from the browser.
4	Cryptographic	SIWE nonces via crypto.getRandomValues(). No predictable nonce sequences.
5	UX	Emergency revocation removes all active permissions. Effective at next block.

No admin keys or privileged roles exist in AgentVaultDepositor.sol. The contract has no owner, no pause function, and no upgrade mechanism. A user's permissions can be revoked only by that user.

4.3 Known Limitations

This system was built for a hackathon. Three limitations apply before any production deployment:

Limitation	Detail
Smart contracts	Unaudited. Third-party audit recommended before mainnet. Estimated cost: \$15-30k.
MockVaults	ERC-4626 simulations only. No live protocol integrations (Aave, Morpho, Pendle, Fluid).
AI reasoning	Venice AI output is probabilistic. Strategy quality depends on DeFiLlama and Tavily data accuracy at request time.

5. User Flow

The full interaction from wallet connection to deployed agents takes six steps and one signature. MetaMask Flask (v13.9.0+) is required, standard MetaMask does not support ERC-7715 permission granting.

Step	Phase	Description
01	AI Strategy	Venice AI fetches live APY data from DeFiLlama and generates a multi-vault allocation plan with reasoning. User reviews vault targets and allocation percentages.
02	Connect Wallet	MetaMask Flask connects. EIP-7702 delegation code is set, upgrading the EOA to a smart account at the same address. No asset migration.
03	Review Skills	Generated agent skill files display in human-readable form. User inspects max amounts, target vaults, and expiry times before proceeding.
04	Grant Permission	One ERC-7715 signature grants scoped permissions to each Worker Agent. No per-transaction signing required after this step.
05	Agent Execution	Orchestrator dispatches Worker Agents in parallel. Each executes Swap -> Approve -> Deposit via 1Shot relay. Gas cost to user: \$0.
06	Live Monitoring	vis.js network graph displays agent activity in real-time. Nodes show permission status, execution progress, and memory log entries. Agent monitors 24/7.

6. Roadmap

Phase	Timeline	Milestones
-------	----------	------------

Phase 1	Hackathon (June 2026)	Sepolia testnet. MockVault ERC-4626 simulations. Proof of architecture: permission enforcement, gasless relay, AI attestation, parallel agent execution, vis.js monitoring graph.
Phase 2	Security (Month 1-2)	Third-party smart contract audit. Formal verification of core invariants: permission bounds, CEI correctness, reentrancy guards. Bug bounty program.
Phase 3	Protocol Integration (Month 3-5)	Adapter pattern for real Aave V3, Compound V3, Curve/Convex, Morpho Blue, and Pendle. Chainlink oracle integration for live price feeds. The Graph indexer for event history.
Phase 4	Production (Month 6+)	Base mainnet deployment. Rebalancing triggers, compounding frequency control, harvest optimization. Governance token design for protocol fee distribution.

7. Conclusion

Vibing Farmer demonstrates that autonomous DeFi agents can be both genuinely useful and cryptographically bounded. EIP-7702 removes the friction of smart account migration. ERC-7715 gives users a tool to express precisely what an agent may do. The smart contract enforces those boundaries unconditionally. Venice AI finds the optimal strategy within them.

The combination produces a new primitive: permission-bounded autonomy. Users define the scope. The AI optimizes within it. The blockchain enforces the boundary. No custody is ever transferred.

The contracts are deployed and verified on Sepolia. The code is open source. The strategy reasoning is attested on-chain. The system is designed to extend: new vault protocols via adapter, new AI providers via the OpenAI-compatible interface, new permission types as ERC-7715 tooling matures.

References

- [1] EIP-7702: Set EOA account code for one transaction. ethereum.org/EIPS/eip-7702
- [2] ERC-7715: Grant Permissions from Wallets. eips.ethereum.org/EIPS/eip-7715
- [3] EIP-7710: Smart contract delegation. eips.ethereum.org/EIPS/eip-7710
- [4] 1Shot API Permissionless Relayer. 1shotapi.com
- [5] Venice AI: Privacy-first AI inference. venice.ai
- [6] DeFiLlama: Open DeFi TVL and APY aggregator. defillama.com
- [7] MetaMask Smart Accounts Kit. docs.metamask.io/smart-accounts-kit
- [8] OpenZeppelin ReentrancyGuard. docs.openzeppelin.com/contracts
- [9] Vibing Farmer source code. github.com/poggufanz/yield-vibing
- [10] Foundry: Fast, portable Ethereum development toolchain. getfoundry.sh